

Algorithm 1 Explanation

Algorithm 2 gives the full formulation of $\text{MAXCOVERAGEPARTIALALIGNMENT}(t, t', \mu)$, invoked in line 2 of Algorithm 1. It uses dynamic programming: it builds a table indexed by pairs of trace positions, fills each entry from previously filled entries, and then recovers an optimal alignment by tracing back through the table. This section states what the algorithm optimizes and then explains how it fills the table and recovers an alignment.

Let $t = \langle n_1, \dots, n_k \rangle$ and $t' = \langle n'_1, \dots, n'_\ell \rangle$ be traces and let $\mu \subseteq N \times N'$ be a node-match relation. Recall from Definition 2.1 that a relation $A \subseteq \{1, \dots, k\} \times \{1, \dots, \ell\}$ pairs positions of the two traces, and that A is a *partial alignment* if it satisfies condition (1), that $(i, j) \in A$ implies $\mu(n_i, n'_j)$, and condition (3), that $(i, j), (i', j') \in A$ and $i < i'$ imply $j \leq j'$; it need not satisfy condition (2), that every node of both traces participates. Let $p(A)$ and $q(A)$ denote the number of positions of t and of t' , respectively, that participate in A :

$$p(A) = |\{i \mid \exists j : (i, j) \in A\}|, \quad q(A) = |\{j \mid \exists i : (i, j) \in A\}|.$$

The *coverage* of A is $\text{cov}(A) = p(A) + q(A)$. Each participating node is counted once, however many pairs it belongs to, which is what makes stuttering free of charge: if two consecutive nodes of t are both paired with the same node of t' , the alignment collects credit for three nodes, not four.

IntentTrace maximizes coverage, and among alignments of maximum coverage maximizes the *weighted participation* $w(A) = q(A) \cdot k + p(A) \cdot \ell$; that is, it computes

$$A^* \in \arg \max_{\preceq_{\text{lex}}} \{(\text{cov}(A), w(A)) \mid A \text{ is a partial alignment of } t \text{ and } t' \text{ under } \mu\}, \quad (1)$$

where $\preceq_{\text{lex}}$ orders pairs lexicographically. Coverage is the primary objective: it is what makes the set of within-pair differences reported by Algorithm 1 minimal in cardinality. The secondary objective is forced rather than chosen. By the Claim below, w is the only alignment-dependent part of $\text{sim}(t, t')$, so maximizing w among the maximum-coverage alignments is exactly maximizing similarity among them, and sim becomes a well-defined function of t , t' , and μ . The two objectives must be ordered this way because they can conflict: w counts a node of the shorter trace for more, so maximizing w alone can favour an alignment of lower coverage.

Claim. *For a trace pair, call a node partnered if it has a μ -partner in the other trace. Let m_t and $m_{t'}$ be the numbers of partnered nodes of t and t' , and let s_t and $s_{t'}$ be the sizes of the two skipped sets of lines 3 and 4 of Algorithm 1. Every node is either partnered or skipped, so $m_t + s_t = k$ and $m_{t'} + s_{t'} = \ell$. If Algorithm 1 is run with an arbitrary partial alignment A in place of A^* , the similarity it computes is*

$$\text{sim}(t, t') = c + \frac{1}{4} \left(\frac{p(A)}{k} + \frac{q(A)}{\ell} \right), \quad c = 1 - \frac{1}{2} \left(\frac{s_t}{k} + \frac{s_{t'}}{\ell} \right) - \frac{1}{4} \left(\frac{m_t}{k} + \frac{m_{t'}}{\ell} \right),$$

where c depends only on t , t' , and μ , not on A . Consequently, among partial alignments of equal coverage, $\text{sim}(t, t')$ is maximal exactly when $p(A)/k + q(A)/\ell$ is maximal, that is, exactly when $w(A) = q(A)k + p(A)\ell$ is maximal.

Proof. The alignment enters Algorithm 1 only through the misordered sets of lines 5 and 6. Line 5 collects the nodes of t that are partnered but do not participate in the alignment, so $|\text{Misordered}_{t \rightarrow t'}| = m_t - p(A)$, and line 6 gives $|\text{Misordered}_{t' \rightarrow t}| = m_{t'} - q(A)$. Substituting the first into the directional score of line 8,

$$\text{sim}(t \rightarrow t') = \frac{k - (s_t + \frac{1}{2}(m_t - p(A)))}{k} = 1 - \frac{s_t}{k} - \frac{m_t}{2k} + \frac{p(A)}{2k},$$

and symmetrically for line 9. Line 10 averages the two, halving each term and so turning the $\frac{1}{2}$ that line 8 assigns to a misordered node into a $\frac{1}{4}$:

$$\text{sim}(t, t') = 1 - \frac{1}{2} \left(\frac{s_t}{k} + \frac{s_{t'}}{\ell} \right) - \frac{1}{4} \left(\frac{m_t}{k} + \frac{m_{t'}}{\ell} \right) + \frac{1}{4} \left(\frac{p(A)}{k} + \frac{q(A)}{\ell} \right),$$

which is the claimed identity. $\square$

Say that a non-empty partial alignment *ends at* the pair (i, j) if (i, j) belongs to it and no pair in it has a larger position in t or a larger position in t' . Every non-empty partial alignment ends at exactly one pair, namely the lexicographically largest pair (i, j) it contains: any other pair (i', j') in it has $i' \leq i$, and then $j' \leq j$ follows from condition (3) if $i' < i$, and from lexicographic maximality if $i' = i$. The entry $V[i, j]$ holds the lexicographic maximum of $(\text{cov}(A), w(A))$ over the partial alignments A that end at (i, j) . When $\mu(n_i, n'_j)$ fails, condition (1) forbids the pair (i, j) outright, no alignment ends there, and the entry keeps its initial value $(-\infty, -\infty)$.

Algorithm 2 fills V in increasing order of i and then of j , computing each entry as follows with component wise addition.

$$V[i, j] = \max_{\preceq_{\text{lex}}} \begin{cases} (2, \ell + k) & \text{start at } (i, j), \\ \max_{j' < j} V[i, j'] + (1, k) & \text{new node of } t', \\ \max_{i' < i} V[i', j] + (1, \ell) & \text{new node of } t, \\ \max_{i' < i, j' < j} V[i', j'] + (2, \ell + k) & \text{new node of each,} \end{cases} \quad (2)$$

Algorithm 2 MAXCOVERAGEPARTIALALIGNMENT(t, t', μ)

Require: Traces $t = \langle n_1, \dots, n_k \rangle$ and $t' = \langle n'_1, \dots, n'_\ell \rangle$; node-match relation μ

Ensure: A partial alignment A^* maximizing (cov, w) lexicographically

Fill the table
1: $V, R, C, P \leftarrow$ tables with all entries $(-\infty, -\infty)$; $back \leftarrow$ table with all entries $\perp$
2: **for** $i \leftarrow 1$ **to** k **and** $j \leftarrow 1$ **to** ℓ , in increasing order of i and then of j **do**
3: **if** $\mu(n_i, n'_j)$ **then**
4: $V[i, j] \leftarrow \max_{\leq_{\text{lex}}} \left(\underbrace{(2, \ell + k)}_{\text{start}}, \underbrace{R[i, j - 1] + (1, k)}_{\text{new node of } t'}, \underbrace{C[i - 1, j] + (1, \ell)}_{\text{new node of } t}, \underbrace{P[i - 1, j - 1] + (2, \ell + k)}_{\text{new node of each}} \right)$
5: $back[i, j] \leftarrow$ the earlier pair attaining the maximum, or $\perp$ if the first argument attains it
6: **end if**
7: $R[i, j] \leftarrow \max_{\leq_{\text{lex}}} (R[i, j - 1], V[i, j])$; $C[i, j] \leftarrow \max_{\leq_{\text{lex}}} (C[i - 1, j], V[i, j])$
8: $P[i, j] \leftarrow \max_{\leq_{\text{lex}}} (P[i - 1, j], P[i, j - 1], V[i, j])$
9: **end for**
Recover an alignment
10: **if** every entry of V is $(-\infty, -\infty)$ **then return** $\emptyset$
11: $A^* \leftarrow \emptyset$; $(i, j) \leftarrow$ the index of an entry of V that is maximal under $\leq_{\text{lex}}$
12: **while** $(i, j) \neq \perp$ **do**
13: $A^* \leftarrow A^* \cup \{(i, j)\}$; $(i, j) \leftarrow back[i, j]$
14: **end while**
15: **return** A^*

The four cases enumerate the ways an alignment can end at (i, j) . Either it is the single pair $\{(i, j)\}$, or it consists of (i, j) together with a smaller partial alignment that ends at an *earlier* pair (i', j') , meaning one with $i' \leq i$, $j' \leq j$, and $(i', j') \neq (i, j)$. Three kinds of earlier pair are possible, matching the last three cases: $i' = i$ with $j' < j$, so that (i, j) contributes a new node of t' while stuttering on n_i ; $i' < i$ with $j' = j$, the mirror image; and $i' < i$ with $j' < j$, contributing a new node of each trace. Conversely, adding (i, j) to any partial alignment ending at an earlier pair yields a partial alignment ending at (i, j) : condition (1) holds because $\mu(n_i, n'_j)$, and condition (3) holds because every pair of the smaller alignment has both of its positions bounded by those of (i', j') , which are in turn bounded by i and j .

Because each table entry is a pair (cov, w) , the amount by which extending an alignment improves it is also a pair, and this is what the increments in Equation (2) record. Coverage rises by one for each node that newly participates. Weighted participation rises by ℓ for a new node of t and by k for a new node of t' , since $w(A) = q(A)k + p(A)\ell$ weights participating nodes of t by ℓ and those of t' by k . A node is new exactly when the extension advances past it: every position of t in the smaller alignment is at most i' , so n_i newly participates precisely when $i' < i$, and symmetrically n'_j precisely when $j' < j$. Hence the increment is $(2, \ell + k)$ for the single pair $\{(i, j)\}$, in which both nodes are new; $(1, k)$ when only n'_j is new; $(1, \ell)$ when only n_i is new; and $(2, \ell + k)$ when both are. These increments depend only on which case applies and not on the smaller alignment itself, so an optimal alignment ending at (i, j) is (i, j) added to an optimal alignment ending at an earlier pair, which is what makes the tabulation correct.

Each of the three max terms in Equation (2) ranges over many earlier entries, and rescanning them for every cell would dominate the cost. Algorithm 2 instead carries the needed values forward in three lookup tables, each holding the lexicographically largest entry seen so far in a region: $R[i, j]$ for row i up to column j , $C[i, j]$ for column j up to row i , and $P[i, j]$ for the rectangle of rows up to i and columns up to j . Each table is updated by a single comparison against $V[i, j]$, namely $R[i, j] = \max(R[i, j - 1], V[i, j])$, $C[i, j] = \max(C[i - 1, j], V[i, j])$ and $P[i, j] = \max(P[i - 1, j], P[i, j - 1], V[i, j])$, with index-0 entries taken as $(-\infty, -\infty)$; shifting each index back by one then supplies exactly the three values that Equation (2) requires. Every cell is therefore filled by a constant number of comparisons, so the table costs $O(k\ell)$ time and space in total. The traces are bounded in length, since loops are unrolled at most once during trace extraction.

Filling the table yields the optimal value but not an alignment attaining it, so the algorithm also records in $back[i, j]$ the earlier pair through which $V[i, j]$ was attained, or $\perp$ if $\{(i, j)\}$ alone attained it. Following these back-pointers from a best entry down to $\perp$ collects a set of pairs that, by the argument above, is a partial alignment attaining Equation (1); if no nodes match under μ , the table stays empty and the algorithm returns $\emptyset$, which is then optimal.